

C++ - Module 01

Allocation mémoire, pointeurs sur membres,
références et instructions switch

Résumé :

Ce document contient les exercices du Module 01 des modules C++.

Version : 11.1

Table des matières

I	Introduction	2
II	Règles générales	3
III	Instructions concernant l'IA	6
IV	Exercice 00 : BraiiiiiiinnnzzzzZ	8
V	Exercice 01 : Moar brainz !	9
VI	Exercice 02 : HI THIS IS BRAIN	10
VII	Exercice 03 : Violence inutile	11
VIII	Exercice 04 : Sed, c'est pour les losers	13
IX	Exercice 05 : Harl 2.0	14
X	Exercice 06 : Filtre à Harl	16
XI	Rendu et évaluation entre pairs	17

Chapitre I

Introduction

Le C++ est un langage de programmation généraliste créé par Bjarne Stroustrup en tant qu'extension du langage C, ou « C with Classes » (source : Wikipedia).

L'objectif de ces modules est de vous initier à la **Programmation Orientée Objet** (Object-Oriented Programming). Ce sera le point de départ de votre parcours C++. De nombreux langages sont recommandés pour apprendre la POO. Nous avons choisi le C++ car il est dérivé de votre vieil ami, le C. Comme il s'agit d'un langage complexe, et afin de garder les choses simples, votre code respectera le standard C++98.

Nous savons que le C++ moderne est très différent à bien des égards. Donc si vous voulez devenir un(e) développeur(se) C++ compétent(e), c'est à vous d'aller plus loin après le 42 Common Core !

Règles générales

Compilation

- Compilez votre code avec `c++` et les flags `-Wall -Wextra -Werror`
- Votre code doit toujours compiler si vous ajoutez le flag `-std=c++98`

Mise en forme et conventions de nommage

- Les répertoires d'exercices seront nommés ainsi : `ex00`, `ex01`, ..., `exn`
- Nommez vos fichiers, classes, fonctions, fonctions membres et attributs comme demandé dans les consignes.
- Écrivez les noms de classes au format **UpperCamelCase**. Les fichiers contenant le code d'une classe porteront toujours le nom de la classe. Par exemple : `ClassName.hpp/ClassName.h`, `ClassName.cpp`, ou `ClassName.hpp`. Ainsi, si vous avez un fichier d'en-tête contenant la définition d'une classe "BrickWall" représentant un mur de briques, il s'appellera `BrickWall.hpp`.
- Sauf indication contraire, chaque message affiché doit se terminer par un caractère de nouvelle ligne et être affiché sur la sortie standard.
- *Adieu la Norminette !* Aucun style de code n'est imposé dans les modules C++. Vous pouvez suivre celui que vous préférez. Mais gardez à l'esprit qu'un code que vos correcteurs pairs ne peuvent pas comprendre est un code qu'ils ne peuvent pas noter. Faites de votre mieux pour écrire un code propre et lisible.

Autorisé/Interdit

Vous ne codez plus en C. Il est temps de passer au C++ ! Par conséquent :

- Vous êtes autorisé(e) à utiliser presque tout ce que propose la bibliothèque standard. Ainsi, plutôt que de vous en tenir à ce que vous connaissez déjà, il serait judicieux d'utiliser autant que possible les versions « C++ » des fonctions C auxquelles vous êtes habitué(e).
- Cependant, vous ne pouvez utiliser aucune autre bibliothèque externe. Cela signifie que le C++11 (et ses formes dérivées) et les bibliothèques Boost sont interdits. Les fonctions suivantes sont également interdites : `*printf()`, `*alloc()` et `free()`. Si vous les utilisez, votre note sera 0, un point c'est tout.
- Notez que, sauf mention contraire explicite, les mots-clés `using namespace <ns_name>` et `friend` sont interdits. Sinon, votre note sera -42.
- Vous êtes autorisé(e) à utiliser la STL uniquement dans les **Modules 08 et 09**. Cela signifie : pas de **Containers** (`vector/list/map`, etc.) et pas d'**Algorithms** (tout ce qui nécessite d'inclure l'en-tête `<algorithm>`) jusque-là. Sinon, votre note sera -42.

Quelques exigences de conception

- Les fuites de mémoire (*memory leaks*) existent aussi en C++. Lorsque vous allouez de la mémoire (en utilisant le mot-clé `new`), vous devez les éviter.
- Du Module 02 au Module 09, vos classes doivent être conçues selon la **Forme Canonique Orthodoxe** (*Orthodox Canonical Form*), sauf mention contraire explicite.

- Toute implémentation de fonction placée dans un fichier d'en-tête (à l'exception des modèles de fonctions/templates) vaut 0 à l'exercice.
- Vous devez pouvoir utiliser chacun de vos en-têtes indépendamment des autres. Ainsi, ils doivent inclure toutes les dépendances dont ils ont besoin. Cependant, vous devez éviter le problème de la double inclusion en ajoutant des *include guards*. Sinon, votre note sera 0.

Lisez-moi

- Vous pouvez ajouter des fichiers supplémentaires si besoin (c'est-à-dire pour scinder votre code). Ces travaux n'étant pas vérifiés par un programme, n'hésitez pas à le faire tant que vous rendez les fichiers obligatoires.
- Parfois, les consignes d'un exercice semblent courtes, mais les exemples peuvent révéler des exigences qui ne sont pas explicitement écrites dans les instructions.
- Lisez chaque module en entier avant de commencer ! Faites-le vraiment.
- Par Odin, par Thor ! Utilisez votre cerveau !!!

■ Concernant le Makefile pour les projets C++, les mêmes règles qu'en C s'appliquent (voir le chapitre de la Norme concernant le Makefile).

■ Vous allez devoir implémenter beaucoup de classes. Cela peut sembler fastidieux, à moins que vous ne sachiez scripter votre éditeur de texte préféré.

■ Vous disposez d'une certaine liberté pour réaliser les exercices. Cependant, respectez les règles obligatoires et ne soyez pas paresseux(se). Vous manqueriez beaucoup d'informations utiles ! N'hésitez pas à vous renseigner sur les concepts théoriques.

Instructions concernant l'IA

● Contexte

Ce projet est conçu pour vous aider à découvrir les briques fondamentales de votre formation à 42.

Pour bien ancrer les connaissances et compétences clés, il est essentiel d'adopter une approche réfléchie quant à l'utilisation des outils et de l'assistance de l'IA.

Un véritable apprentissage des fondamentaux exige un réel effort intellectuel — à travers le défi, la répétition et les échanges d'apprentissage entre pairs (*peer-learning*).

Pour un aperçu plus complet de notre positionnement sur l'IA — en tant qu'outil d'apprentissage, dans le cadre de la formation 42, et en tant qu'attente sur le marché de l'emploi — merci de vous référer à la FAQ dédiée sur l'intranet.

● Message principal

- ➡ Construisez des bases solides sans raccourcis.
- ➡ Développez réellement vos compétences techniques et humaines (*tech & power skills*).
- ➡ Vivez un vrai *peer-learning*, apprenez à apprendre et à résoudre de nouveaux problèmes.
- ➡ Le chemin d'apprentissage compte plus que le résultat.
- ➡ Renseignez-vous sur les risques associés à l'IA, et développez des pratiques de contrôle efficaces ainsi que des contre-mesures pour éviter les pièges courants.

● Règles pour l'apprenant :

- Vous devez appliquer votre raisonnement aux tâches qui vous sont confiées, particulièrement avant de vous tourner vers l'IA.

- Vous ne devez pas demander de réponses directes à l'IA.
- Vous devez vous renseigner sur l'approche globale de 42 concernant l'IA.

● Résultats de la phase :

Au cours de cette phase fondamentale, vous obtiendrez les résultats suivants :

- Acquérir de bonnes bases techniques et de codage.
- Comprendre pourquoi et comment l'IA peut être dangereuse durant cette phase.

● Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos projets. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne perdez pas votre temps (ni le nôtre) juste pour démontrer que l'IA peut résoudre le problème donné.
- Apprendre à 42, ce n'est pas connaître la réponse — c'est développer la capacité à en trouver une. L'IA vous donne la réponse directement, mais cela vous empêche de construire votre propre raisonnement. Or, raisonner prend du temps, demande des efforts, et implique l'échec. Le chemin vers la réussite n'est pas censé être facile.
- Gardez à l'esprit que pendant les examens, l'IA n'est pas disponible — pas d'internet, pas de smartphones, etc. Vous réaliserez vite si vous avez trop compté sur l'IA dans votre apprentissage.
- Le *peer-learning* vous expose à des idées et approches différentes, améliore vos compétences relationnelles et votre capacité à penser de façon divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors n'ayez pas peur — parlez, posez des questions, apprenez ensemble !
- Oui, l'IA fera partie du cursus — à la fois comme outil d'apprentissage et comme sujet à part entière. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Pour en savoir plus sur notre approche en crescendo, vous la retrouverez dans la documentation disponible sur l'intranet.

✓ Bonne pratique :

Je suis bloqué(e) sur une nouvelle notion. Je demande à quelqu'un à côté comment il/elle a abordé le problème. On discute pendant 10 minutes — et soudain, ça fait tilt. Je comprends.

✗ Mauvaise pratique :

J'utilise l'IA en secret, je copie du code qui a l'air correct. Pendant l'évaluation entre pairs, je ne peux rien expliquer. J'échoue. Pendant l'examen — pas d'IA — je suis à nouveau bloqué(e). J'échoue.

Exercice 00 : BraiiiiiiinnnzzzzZ

Exercice : 00	
BraiiiiiiinnnzzzzZ	
Répertoire :	ex00/
Fichiers à rendre :	Makefile, main.cpp, Zombie.{h, hpp}, Zombie.cpp, newZombie.cpp, randomChump.cpp
Interdit :	Aucune

D'abord, implémentez une classe **Zombie**. Elle possède un attribut privé de type `string` nommé `name`.

Ajoutez une fonction membre `void announce( void );` à la classe `Zombie`. Les zombies s'annoncent de la façon suivante :

N'affichez pas les chevrons (`<` et `>`). Pour un zombie nommé `Foo`, le message serait :

Ensuite, implémentez les deux fonctions suivantes :

- `Zombie* newZombie( std::string name );`

Cette fonction crée un zombie, le nomme, et le retourne afin que vous puissiez l'utiliser en dehors de la portée de la fonction.

- `void randomChump( std::string name );`

Cette fonction crée un zombie, le nomme, et le fait s'annoncer.

Alors, quel est le véritable but de cet exercice ? Vous devez déterminer dans quel cas il est préférable d'allouer les zombies sur la pile (*stack*) ou sur le tas (*heap*).

Les zombies doivent être détruits lorsque vous n'en avez plus besoin. Le destructeur doit afficher un message avec le nom du zombie, à des fins de débogage.

Exercice 01 : Moar brainz !

Exercice : 01	
Moar brainz !	
Répertoire :	<i>ex01/</i>
Fichiers à rendre :	Makefile, main.cpp, Zombie.{h, hpp}, Zombie.cpp, zombieHorde.cpp
Interdit :	Aucune

Il est temps de créer une **horde de Zombies** !

Implémentez la fonction suivante dans le fichier approprié :

Elle doit allouer N objets Zombie en une seule allocation. Ensuite, elle doit initialiser les zombies, en donnant à chacun le nom passé en paramètre. La fonction retourne un pointeur vers le premier zombie. Implémentez vos propres tests pour vous assurer que votre fonction `zombieHorde()` fonctionne comme prévu. Essayez d'appeler `announce()` pour chacun des zombies.

N'oubliez pas d'utiliser `delete` pour désallouer tous les zombies et vérifiez l'absence de **fuites de mémoire**.

Exercice 02 : HI THIS IS BRAIN

Exercice : 02	
HI THIS IS BRAIN	
Répertoire :	ex02/
Fichiers à rendre :	Makefile, main.cpp
Interdit :	Aucune

Écrivez un programme qui contient :

- Une variable `string` initialisée à "HI THIS IS BRAIN".
- `stringPTR` : un pointeur vers la string.
- `stringREF` : une référence vers la string.

Votre programme doit afficher :

- L'adresse mémoire de la variable `string`.
- L'adresse mémoire détenue par `stringPTR`.
- L'adresse mémoire détenue par `stringREF`.

Puis :

- La valeur de la variable `string`.
- La valeur pointée par `stringPTR`.
- La valeur pointée par `stringREF`.

C'est tout — aucun piège. Le but de cet exercice est de démystifier les **références**, qui peuvent sembler totalement nouvelles. Bien qu'il existe de petites différences, il s'agit simplement d'une autre syntaxe pour quelque chose que vous faites déjà : la manipulation d'adresses.

Exercice 03 : Violence inutile

Exercice : 03	
Violence inutile	
Répertoire :	ex03/
Fichiers à rendre :	Makefile, main.cpp, Weapon.{h, hpp}, Weapon.cpp, HumanA.{h, hpp}, HumanA.cpp, HumanB.{h, hpp}, HumanB.cpp
Interdit :	Aucune

Implémentez une classe **Weapon** qui possède :

- Un attribut privé `type`, de type `string`.
- Une fonction membre `getType()` qui retourne une référence constante vers `type`.
- Une fonction membre `setType()` qui définit `type` à partir de la nouvelle valeur passée en paramètre.

Créez maintenant deux classes : **HumanA** et **HumanB**. Elles possèdent toutes deux une `Weapon` et un `name`. Elles ont aussi une fonction membre `attack()` qui affiche (sans les chevrons) :

`HumanA` et `HumanB` sont presque identiques, à l'exception de ces deux petits détails :

- Alors que **HumanA** reçoit la `Weapon` dans son constructeur, **HumanB** n'en reçoit pas.
- **HumanB** n'a pas forcément toujours une arme, alors que **HumanA** sera toujours armé.

Si votre implémentation est correcte, l'exécution du code suivant affichera une attaque avec "crude spiked club" suivie d'une seconde attaque avec "some other type of club", pour les deux cas de test :

N'oubliez pas de vérifier l'absence de **fuites de mémoire**.

■ Dans quel cas pensez-vous qu'il serait préférable d'utiliser un pointeur vers Weapon ? Et une référence vers Weapon ? Pourquoi ? Réfléchissez-y avant de commencer cet exercice.

Exercice 04 : Sed, c'est pour les losers

Exercice : 04	
Sed, c'est pour les losers	
Répertoire :	<code>ex04/</code>
Fichiers à rendre :	<code>Makefile, main.cpp, *.cpp, *.{h, hpp}</code>
Interdit :	<code>std::string::replace</code>

Créez un programme qui prend trois paramètres, dans l'ordre suivant : un nom de fichier et deux strings, `s1` et `s2`.

Il doit ouvrir le fichier `<filename>` et copier son contenu dans un nouveau fichier `<filename>.replace`, en remplaçant chaque occurrence de `s1` par `s2`.

Utiliser les fonctions de manipulation de fichiers du C est interdit et sera considéré comme de la triche. Toutes les fonctions membres de la classe `std::string` sont autorisées, sauf `replace`. Utilisez-les intelligemment !

Bien entendu, gérez les entrées et erreurs inattendues. Vous devez créer et rendre vos propres tests pour vous assurer que votre programme fonctionne comme prévu.

Exercice 05 : Harl 2.0

Exercice : 05	
Harl 2.0	
Répertoire :	ex05/
Fichiers à rendre :	Makefile, main.cpp, Harl.{h, hpp}, Harl.cpp
Interdit :	Aucune

Connaissez-vous Harl ? Nous le connaissons tous, n'est-ce pas ? Au cas où non, voici le genre de commentaires que Harl fait. Ils sont classés par niveaux :

- **"DEBUG"** : Les messages de debug contiennent des informations contextuelles. Ils sont principalement utilisés pour le diagnostic de problèmes.
Exemple : « J'adore avoir du bacon supplémentaire sur mon burger 7XL-double-cheese-triple-pickle-spécial-ketchup. C'est vrai ! »
- **"INFO"** : Ces messages contiennent des informations détaillées. Ils sont utiles pour suivre l'exécution du programme en environnement de production.
Exemple : « Je n'arrive pas à croire qu'ajouter du bacon coûte plus cher. Vous n'avez pas mis assez de bacon dans mon burger ! Si vous l'aviez fait, je n'en redemanderais pas ! »
- **"WARNING"** : Les messages d'avertissement indiquent un problème potentiel dans le système. Cependant, il peut être géré ou ignoré.
Exemple : « Je pense que je mérite d'avoir du bacon supplémentaire gratuitement. Je viens ici depuis des années, alors que vous avez commencé à travailler ici le mois dernier. »
- **"ERROR"** : Ces messages indiquent qu'une erreur irrécupérable s'est produite. Il s'agit généralement d'un problème critique nécessitant une intervention manuelle.
Exemple : « C'est inacceptable ! Je veux parler au responsable maintenant. »

Vous allez automatiser Harl. Ce ne sera pas difficile puisqu'il dit toujours les mêmes choses. Vous devez créer une classe **Harl** avec les fonctions membres privées suivantes :

- `void debug( void );`
- `void info( void );`
- `void warning( void );`
- `void error( void );`

Harl possède aussi une fonction membre publique qui appelle les quatre fonctions membres ci-dessus selon le niveau passé en paramètre :

Le but de cet exercice est d'utiliser des **pointeurs sur fonctions membres** (*pointers to member functions*). Ce n'est pas une suggestion. Harl doit se plaindre sans utiliser une forêt de `if/else if/else`. Il ne réfléchit pas à deux fois !

Créez et rendez des tests montrant que Harl se plaint beaucoup. Vous pouvez utiliser les exemples de commentaires listés ci-dessus dans l'énoncé, ou choisir d'utiliser vos propres commentaires.

Exercice 06 : Filtre à Harl

Exercice : 06	
Filtre à Harl	
Répertoire :	<i>ex06/</i>
Fichiers à rendre :	Makefile, main.cpp, Harl.{h, hpp}, Harl.cpp
Interdit :	Aucune

Parfois, vous ne voulez pas prêter attention à tout ce que dit Harl. Implémentez un système pour filtrer ce que dit Harl selon les niveaux de log que vous souhaitez écouter.

Créez un programme qui prend en paramètre l'un des quatre niveaux. Il affichera tous les messages de ce niveau et au-dessus. Par exemple :

Bien qu'il existe plusieurs façons de gérer Harl, l'une des plus efficaces est de le **SWITCHer** (l'éteindre).

Donnez le nom `harlFilter` à votre exécutable.

Vous devez utiliser, et peut-être découvrir, l'instruction **switch** dans cet exercice.

■ Vous pouvez valider ce module sans faire l'exercice 06.

Chapitre XI

Rendu et évaluation entre pairs

Rendez votre devoir dans votre dépôt **Git** comme d'habitude. Seul le travail présent dans votre dépôt sera évalué pendant la soutenance. N'hésitez pas à revérifier les noms de vos dossiers et fichiers pour vous assurer qu'ils sont corrects.

Pendant l'évaluation, une brève **modification du projet** peut occasionnellement être demandée. Cela peut impliquer un changement mineur de comportement, quelques lignes de code à écrire ou réécrire, ou une fonctionnalité facile à ajouter.

Bien que cette étape puisse **ne pas s'appliquer à tous les projets**, vous devez y être préparé(e) si elle est mentionnée dans les consignes d'évaluation.

Cette étape vise à vérifier votre compréhension réelle d'une partie spécifique du projet. La modification peut être réalisée dans l'environnement de développement de votre choix (par ex., votre configuration habituelle), et devrait être réalisable en quelques minutes — sauf si un délai spécifique est défini dans le cadre de l'évaluation.

Vous pouvez, par exemple, être invité(e) à effectuer une petite mise à jour d'une fonction ou d'un script, modifier un affichage, ou ajuster une structure de données pour stocker de nouvelles informations, etc.

Les détails (portée, cible, etc.) seront précisés dans les **consignes d'évaluation** et peuvent varier d'une évaluation à l'autre pour un même projet.